

Tries

Trie ఒక hash map యొక్క మెరుగైన రూపం కాదు. అది **ఒక ప్రత్యేక ప్రశ్నకి** మాత్రమే మెరుగైనది – మరియు మిగతా ప్రతి ప్రశ్నకీ **నాసిరకం**.

మొదటిది – ఒక్క **end** గుర్తు మర్చిపోతే, నిజమైన ఉపసర్గల మీద **100.0%** తప్పు, కానీ యాదృచ్ఛిక strings మీద **3.78%**. రెండోది – ఉపసర్గ వెతుకులాటకి trie **208x** వేగం, మరియు కచ్చితమైన వెతుకులాటకి **Set** **19.9x** వేగం. మూడోది – ఆ వేగం **30x** memory తో కొన్నది, మరియు 26-గళ్ళ array వాడితే అది **96.2%** ఖాళీ.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి సంఖ్యా నిజంగా `node lo run చేసినదే`, మరియు అవన్నీ ఒక నిజమైన నిఘంటువు మీద – `/usr/share/dict/words`, అందులో a-z పదాలు **2,34,454**, మొత్తం అక్షరాలు **22,48,642**.

TRIE ఈ SERIES LO ఏకైక STRUCTURE – దీనికి ఒక ఒక ఒక్క కారణం ఉంది

Hash map: కచ్చితమైన key → విలువ. **వేగం, చౌక.**

Trie: కచ్చితమైన key → విలువ. **నెమ్మది, ఖరీదు.**

అలాంటప్పుడు trie ఎందుకు? ఎందుకంటే hash map ఒక ప్రశ్నకి జవాబు ఇవ్వలేదు: **"'pre' తో మొదలయ్యేవి ఏవి?"** – దానికి hash map ప్రతి key నీ చూడాల్సిందే.

కాబట్టి trie కి ఒకే ఒక్క సమర్థన: ఉపసర్గ (prefix). అది మే problem lo లేకపోతే – trie వద్దు. §7 lo రెండు వైపులా కొలుస్తాం.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Trie ఒక ఆకారం, ఒక structure కాదు
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: ఒక bit

3. Step – trie కట్టడం
4. మొదటి విరుపు – 3.78%, ఆపై 100.0%
5. Step – `end` మరియు `startsWith` వేరు

Part 3 – రెండో విరుపు: దీనికి వేగం

6. Step – ఉపసర్గ వెతుకులాట
7. రెండో విరుపు – 208× వేగం, మరియు 19.9× నెమ్మది

Part 4 – మూడో విరుపు: ఖరీదు

8. Step – nodes ఎన్ని?
9. మూడో విరుపు – 30× memory, 96.2% ఖాళీ
10. Step – తీసేయడం ఎందుకు కష్టం

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code
12. 3,600 tries · mutation testing

13. Trie ఎప్పుడు వద్దు
14. Follow-ups – radix tree, Aho-Corasick, DAWG
15. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Trie ఒక ఆకారం, ఒక structure కాదు

Trie lo విలువలు nodes lo ఉండవు – అవి దారిలో ఉంటాయి.

"cat" అనే పదం ఎక్కడా నిల్వ ఉండదు. `c → a → t` అనే మూడు అంచుల దారి ఉంటుంది, మరియు చివరి node మీద ఒక గుర్తు: "ఇక్కడ ఒక పదం ముగిసింది."

దీని నుంచి రెండు విషయాలు వెంటనే వస్తాయి

1. ఉపసర్గలు ఉచితంగా పంచుకోబడతాయి. cat, car, card – ca ఒక్కసారే నిల్వ.

2. "ఒక పదం ముగిసింది" అనేది ప్రత్యేకంగా గుర్తుపెట్టాలి. ఎందుకంటే ca దగ్గర ఒక node ఉంది, కానీ ca ఒక పదం కాదు.

రెండోది ఒక్క boolean. మరియు §4 lo చూస్తాం – ఆ ఒక్క boolean మర్చిపోతే తప్పుల రేటు 100.0%.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఉపసర్గ ప్రశ్నలు నిజంగా ఉన్నాయా?	లేకపోతే trie వద్దు – §7 lo Set 19.9x వేగం
ఎన్ని పదాలు, ఎంత పొడవు?	§9 – 2.34 లక్షల పదాలకి trie 150 MB, Set 5 MB
అక్షరమాల (alphabet) ఏమిటి?	a-z ఆ, Unicode ఆ? §9 – 26-గళ్ళ array Unicode కి పనిచేయదు
తేసేయడం అవసరమా?	అవసరమైతే §10 – ఇది చాలా tricky, మరియు ఇక్కడ చాలా bugs
ఎన్ని సూచనలు చూపించాలి?	10 చాలంటే traversal ముందే ఆపొచ్చు – §7 lo అదే కొలిచాను
Trie మారుతూ ఉంటుందా, స్థిరమా?	స్థిరమైతే §14 – DAWG లేదా sorted array + binary search చౌక

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఉపసర్గ ప్రశ్నలు నిజంగా ఉన్నాయా, లేక కచ్చితమైన వెతుకులాట మాత్రమేనా?"

ఎందుకంటే §7 lo కొలిచినది ఇదే, మరియు జవాబు రెండు వైపులా పెద్దది:

ఉపసర్గ → trie 208× వేగం (831 ms → 4 ms) · కచ్చితం → Set 19.9× వేగం (18 ms vs 358 ms).

మరియు trie 30× memory తీసుకుంటుంది. కాబట్టి తప్పు ఎంపిక చేస్తే నెమ్మది మరియు ఖరీదు.

Part 2 – మొదటి విరుపు: ఒక bit

3. Step – trie కట్టడం

```
const node = () => ({ ch: new Map() });
function insert(root, word) {
  let n = root;
  for (const c of word) {
    if (!n.ch.has(c)) n.ch.set(c, node());
    n = n.ch.get(c);
  }
}
function has(root, word) {
  let n = root;
  for (const c of word) {
    if (!n.ch.has(c)) return false;
    n = n.ch.get(c);
  }
  return true; // దారి ఉంది → పదం ఉంది
}
```

నిఘంటువు మొత్తం చేర్చి పరీక్షిద్దాం: `has("apple")` ✓ · `has("zzzz")` ✗ · `has("computer")` ✓ · `has("qwxyz")` ✗.

నాలుగూ సరైనవి.

4. మొదటి విరుపు – 3.78%, అపై 100.0%

`end` గుర్తు లేకపోతే – "ఈ పదం నిఘంటువులో ఉందా?"

ఏ strings పరీక్షించాం	పరీక్షించినవి	"ఉంది" అన్నవి	%
యాదృచ్ఛిక అక్షర strings	1,96,595	7,430	3.78%
నిజమైన పదాల *ఉపసర్గలు*	1,23,047	1,23,047	100.00%

ఒకే BUG. ఒక DATA మీద 3.78%, ఇంకే DATA మీద 100.0%.

```
"dep" ← "depasturation" యొక్క ఉపసర్గ · "op" ← "opprobriously" · "isal" ←  
"isallotherm" · "electrob" ← "electrobath"
```

ఇవి పదాలు కావు. కానీ trie lo వీటికి **దారి ఉంది**, ఎందుకంటే పొడవైన పదాలు వాటి గుండా వెళ్ళాయి. end గుర్తు లేకపోతే "దారి ఉంది" అంటే "పదం ఉంది".

మరియు 100.0% – ఒక్కటి కూడా తప్పలేదు, 1,23,047 lo.

ఇక్కడ హారం (DENOMINATOR) ఒక అభిప్రాయం కాదు – అది మీ నిజమైన TRAFFIC

యాదృచ్ఛిక strings మీద ఇది **3.78%** – "అరుదైన edge case" అనిపిస్తుంది.

కానీ trie ఎక్కడ వాడతారు? **Autocomplete lo.** అక్కడ ప్రతి keystroke ఒక **ఉపసర్గ**. user d , de , dep అని టైప్ చేస్తున్నాడు.

అంటే మీ నిజమైన inputs 100% వరుసలోనివి, 3.78% వరుసలోనివి కావు.

Deep 10 lo హారం మారితే 1.38% → 35.4% · **Deep 11 lo** 9.21% → 81.9% · ఇక్కడ 3.78% → **100.0%**.
ప్రతిసారి అదే ప్రశ్న: "నా నిజమైన inputs ఏ వరుసలో ఉంటాయి?"

5. Step – end మరియు startsWith వేరు

```
const node = () => ({ ch: new Map(), end: false }); // ← ఒక boolean  
  
function has(root, word) { // కచ్చితమైన పదం  
  const n = walk(root, word);  
  return n !== null && n.end; // ← దారి *మరియు* గుర్తు  
}  
  
function startsWith(root, prefix) { // ఈ ఉపసర్గతో ఏదైనా ఉందా  
  return walk(root, prefix) !== null; // ← దారి చాలు, గుర్తు వద్దు  
}
```

ఇవి రెండు వేర్వేరు ప్రశ్నలు, మరియు రెండింటికీ TRIE కావాలి

has("app") → **false** (అది పదం కాదు) · startsWith("app") → **true** (apple , apply ఉన్నాయి).

ఒకే పదాలు, ఒకే trie, ఒకే node – **వేర్వేరు జవాబులు**. మరియు తేడా సరిగ్గా ఒక && n.end .

§12 lo రెండు mutations ఇదే: has lo end తీసేస్తే **120/120**, startsWith lo end కలిపితే **120/120**.

Part 3 – రెండో విరుపు: దేనికి వేగం

6. Step – ఉపసర్గ వెతుకులాట

"pre" తో మొదలయ్యే 10 పదాలు చూపించు.

Set లేదా array తో: ప్రతి పదాన్నీ చూసి startsWith చేయాలి. Trie తో: 3 nodes నడిచి, అక్కడి నుంచి కిందికి 10 దిరికేవరకు.

7. రెండో విరుపు – 208x వేగం, మరియు 19.9x నెమ్మది

2,34,454 పదాలు · 2,000 ఉపసర్గ queries · ఒక్కోదానికి 10 సూచనలు

Array ని scan చేయడం : 831 ms (19,461 ఫలితాలు)
Trie : 4 ms (19,461 ఫలితాలు)
→ 208x వేగం

అదే నిఘంటువు · 5,00,000 *కచ్చితమైన* queries

Set : 18 ms
Trie : 358 ms
→ Set 19.9x వేగం

TRIE ఒక మెరుగైన HASH MAP కాదు. అది ఒక వేరే STRUCTURE.

208x వేగం మరియు 19.9x నెమ్మది – ఒకే data, ఒకే structures, రెండు వేర్వేరు ప్రశ్నలు.

కారణం సూటిది: Set.has("computer") ఒక hash లెక్క మరియు ఒక పోలిక. Trie కి ఎనిమిది Map lookups – ఒక్కో అక్షరానికి ఒకటి, ఒక్కొక్కటి ఒక pointer దూకు.

Trie యొక్క $O(m)$ hash map యొక్క $O(1)$ కంటే నెమ్మది, ఎందుకంటే ఆ m నిజమైన memory దూకులు.

మరియు ఉపసర్గ ప్రశ్నలో hash map యొక్క $O(1)$ వర్తించనే వర్తించదు – అక్కడ అది $O(n)$, $n = 2.34$ లక్షలు.

Part 4 – మూడో విరుపు: ఖరీదు

8. Step – nodes ఎన్ని?

Trie ఎంత పెద్దది అవుతుంది? ఒక node ప్రతి ప్రత్యేక ఉపసర్గకి – కాబట్టి లెక్కపెడదాం.

9. మూడో విరుపు – 30× memory, 96.2% ఖాళీ

2,34,454 పదాలు · మొత్తం అక్షరాలు 22,48,642

Trie nodes : 7,58,880
నిజంగా వాడిన పిల్లల గళ్ళు : 7,58,879
ఒక node కి సగటున పిల్లలు : 1.00
26-గళ్ళ array అయితే గళ్ళు : 1,97,30,880
→ 3.8% మాత్రమే వాడతాం, 96.2% ఖాళీ

సగటున 1.00 పిల్లలు – మరియు ఇది యాదృచ్ఛికం కాదు

Trie ఒక tree. Tree lo అంచులు ఎప్పుడూ nodes – 1. కాబట్టి ఒక node కి సగటు పిల్లలు ఎప్పుడూ దాదాపు సరిగ్గా 1 – ఏ పద జాబితా అయినా సరే.

$7,58,879 = 7,58,880 - 1$. ఖచ్చితంగా.

అంటే 26-గళ్ళ array ఎప్పుడూ ~96% ఖాళీగా ఉంటుంది – ఇది ఈ నిఘంటువు యొక్క లక్షణం కాదు, tries యొక్క లక్షణం.

heap usage · ఒక్కో ప్రక్రియలో విడిగా కొలిచినది

Set (పదాలన్నీ) : 5 MB
Trie · Map పిల్లలు : 150 MB → 30×
Trie · object పిల్లలు : 151 MB → 30×
Trie · 26-గళ్ళ array : 200 MB → 40×

26-గళ్ళ ARRAY "వేగం కోసం" - కానీ అది 1.33x ఎక్కువ MEMORY

పాఠ్యపుస్తకాలు `new Array(26)` చెబుతాయి, ఎందుకంటే `ch[c - 'a']` ఒక Map lookup కంటే వేగం.

కొలత: **200 MB vs 150 MB**. మరియు 96.2% ఆ గళ్ళు `null`.

మరియు అది **a-z కి మాత్రమే** పనిచేస్తుంది. తెలుగు, ఎమోజీ, అంకెలు, hyphen, apostrophe – ఏదైనా ఒకటి వస్తే `c.charCodeAt(0) - 97` పరిధి దాటుతుంది, మరియు అది **crash కాదు**: `undefined` లేదా తప్పు గడి.

Map వాడండి. 25% తక్కువ memory, ఏ అక్షరమాల అయినా సరే, మరియు వేగం తేడా 57 యొక్క 208x ముందు కనిపించదు.

10. Step – తీసేయడం ఎందుకు కష్టం

`insert` మరియు `has` ఐదేసి పంక్తులు. `remove` కాదు – ఎందుకంటే **nodes పంచుకోబడతాయి**.

"apple" తీసేయాలంటే `app` యొక్క `nodes` ని తొక్కుతాడు, ఎందుకంటే "app" ఇంకా ఒక పదం కావచ్చు, లేదా "apply" ఆ దారిలో ఉండొచ్చు.

```
function remove(root, word) {
  const path = [];
  let n = root;
  for (const c of word) {
    if (!n.ch.has(c)) return false;           // పదమే లేదు
    path.push([n, c]);
    n = n.ch.get(c);
  }
  if (!n.end) return false;                   // దారి ఉంది, కానీ అది ఒక పదం కాదు
  n.end = false;
  for (let i = path.length - 1; i >= 0; i--) { // ← *కింది నుంచి* పైకి
    const [parent, c] = path[i];
    const child = parent.ch.get(c);
    if (child.ch.size > 0 || child.end) break; // ఇంకెవరో వాడుతున్నారు – ఆగు
    parent.ch.delete(c);
  }
  return true;
}
```

ఈ పదిహేను పంక్తుల్లో నాలుగు LOAD-BEARING

`if (!n.end) return false` – "apple" ఉన్నప్పుడు `remove("app")` ని ఆపుతుంది. లేకపోతే ఏమీ లేని పదాన్ని "తీసేశాను" అంటుంది. → **120/120**

`child.ch.size > 0` – "apply" ఆ దారిలో ఉంటే ఆపుతుంది. → **120/120**

`child.end` – "app" కూడా ఒక పదం అయితే ఆపుతుంది. → **38/120** (§12 lo ఎందుకే)

కింది నుంచి పైకి – పై నుంచి తుడిస్తే మొదటి node దగ్గరే ఆగిపోతుంది, మరియు **చచ్చిన nodes మిగులుతాయి**.
→ **120/120**

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code

```
// ఈ ఉపసర్గతో మొదలయ్యే పదాలు (నిఘంటువు క్రమంలో, limit వరకు)
function withPrefix(root, prefix, limit = Infinity) {
  const start = walk(root, prefix);
  const out = [];
  if (!start) return out;
  const go = (n, p) => {
    if (out.length >= limit) return;
    if (n.end) out.push(p);
    for (const c of [...n.ch.keys()].sort()) {
      if (out.length >= limit) return;
      go(n.ch.get(c), p + c);
    }
  };
  go(start, prefix);
  return out;
}
```

మూడు పంక్తులూ ముఖ్యం. `limit` తనిఖీ లేకపోతే `withPrefix("a")` మొత్తం `a` పదాలనీ కూడబెడుతుంది – §7 యొక్క 4 ms అది **10 దగ్గర ఆగడం** వల్లే.

12. 3,600 tries • mutation testing

పద్నాలుగు నియమాలు: చేర్చినవన్నీ దొరుకుతాయి, పదం కాని ఉపసర్గలు `has` కీ `false`, కానీ `startsWith` కీ `true`, లేని పదాలు దొరకవు, `withPrefix` = filter చేసి sort చేసినది, ప్రతి ఫలితమూ ఆ ఉపసర్గతోనే, నకళ్ళు లేకుండా, `limit` ఇస్తే సరిగ్గా మొదటివే, `has` సరిగ్గా `word.length` అడుగులు, తీసేసినది పోతుంది, మిగతావి ఉంటాయి, తీసేశాక `nodes` సరిగ్గా తగ్గుతాయి, లేని పదానికి `false`, trie మారదు, పదం కాని ఉపసర్గకీ `false`, trie మారదు, మళ్ళీ చేర్చితే అంతా తిరిగి వస్తుంది, `nodes` = ప్రత్యేక ఉపసర్గలు.

300 runs • 3,600 tries • 1,95,141 ఉపసర్గ queries
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

tries	3,600	
realWords	1,288	← నిజమైన నిఘంటువు నుంచి
sharedPrefix	1,369	← "a", "ap", "app", "appl" లాంటివి
prefixIsWord	14,751	← ఉపసర్గ *కూడా* ఒక పదమైన సందర్భాలు
removes	3,600	
removeMissing	3,600	

removePrefix 3,207 ← పదం *కాని* ఉపసర్గని తీసేయాలని చూసినవి
emptyStr 627

==== mutation testing ====

మార్పు లేని code → 0/300 విఫలం
has() lo `end` చూడకపోతే (\$4 యొక్క bug) → 120/120 విఫలం
ఉదా: "" పదం కాదు, కానీ has() true (w="ad")
insert lo `n.end = true` తీసేస్తే → 120/120 విఫలం
startsWith lo `end` కూడా చూస్తే → 120/120 విఫలం
ఉదా: startsWith("") false, కానీ "ad" ఉంది
withPrefix lo `n.end` చూడకుండా ప్రతి node నీ ఇస్తే → 120/120 విఫలం
ఉదా: ["a","ad","adc","adcc","adccc","adccd"] ≠ ["ad","adc","adccc","adccd"]
withPrefix lo పిల్లల keys ని sort చేయకపోతే → 120/120 విఫలం
remove lo `child.end` తనిఖీ తీసేస్తే → 38/120 విఫలం
ఉదా: "cac" తీసేస్తే "c" కూడా పోయింది
remove lo `child.ch.size > 0` తనిఖీ తీసేస్తే → 120/120 విఫలం
ఉదా: "adccc" తీసేస్తే "adccd" కూడా పోయింది
remove lo `if (!n.end) return false` తీసేస్తే → 120/120 విఫలం
ఉదా: "a" ఒక పదం కాదు, కానీ remove() true
remove lo దారిని *పై నుంచి* కిందికి తుడిస్తే → 120/120 విఫలం
ఉదా: "adccc" తీసేశాక nodes 7 ≠ 6 (చచ్చిన nodes మిగిలాయా?)
walk lo `n.ch.has(c)` కి బదులు `n.ch.get(c)` నే వాడితే → 119/120 విఫలం

ఈ పట్టికలో మొదట రెండు MUTATIONS 0/120 గా బతకాయి

if (!n.end) return false తీసేస్తే – 0/120. నా fuzz remove ని ఎప్పుడూ నిజమైన పదాల మీదే పిలిచింది. "దారి ఉంది కానీ పదం కాదు" అనే సందర్భం – సరిగ్గా ఆ పంక్తి కాపాడేది – నా data lo లేనే లేదు.

దారిని పై నుంచి తుడిస్తే – 0/120. నా నియమాలు countNodes ని తీసేసి, మళ్ళీ చేర్చాక మాత్రమే చూశాయి. ఏమీ తీసేయకపోతే, మళ్ళీ చేర్చినా ఏమీ కలవదు, మరియు లెక్క సరిపోతుంది. చచ్చిన nodes మధ్యలో ఉన్నాయి, మరియు నేను ఆ క్షణంలో చూడలేదు.

రెండు కొత్త నియమాలు, రెండూ 120/120

(1) "తీసేసిన వెంటనే nodes = మిగిలిన పదాల ప్రత్యేక ఉపసర్గలు" – ఇది చచ్చిన nodes ని ఆ క్షణంలోనే పట్టుకుంటుంది.

(2) "పదం కాని ఉపసర్గని remove చేస్తే false, మరియు ఏమీ మారకూడదు" – 120 runs lo 3,207 సార్లు నడిచింది.

child.end mutation 38/120 దగ్గరే ఆగింది, మరియు అది నిజాయితీగా ఆగింది: అది విఫలం కావాలంటే ఒక పదం ఇంకో పదానికి ఖచ్చితమైన ఉపసర్గ కావాలి, మరియు పొడవైనదే తీసేయాలి. ఆ రెండూ కలిసి రావడం 120 runs lo 38 సార్లు జరిగింది.

మళ్ళీ అదే పాఠం: బలహీనమైన mutation ఫలితం అంటే బలహీనమైన code కాదు – బలహీనమైన data.

13. Trie ఎప్పుడు వద్దు

ఈ doc మొత్తం trie ని సమర్థిస్తోందని అనిపించొచ్చు. కాదు. కొలతలు చెప్పేది ఇది:

మీ ప్రశ్న	వాడాల్సినది	ఎందుకు
"ఈ పదం ఉందా?"	Set	19.9× వేగం, 30× తక్కువ memory
"ఈ ఉపసర్గతో ఏమున్నాయి?"	Trie	208× వేగం
"ఈ పదం ఉందా, మరియు జాబితా స్థిరం?"	sorted array + binary search	Deep 01 – memory దాదాపు సున్నా
"మధ్యలో ఈ భాగం ఉన్నవి ఏవి?"	Trie కాదు – suffix structures	Trie ఉపసర్గలకే
Unicode / ఏ భాష అయినా	Trie + Map	26-గళ్ళ array పనిచేయదు (§9)

2.34 లక్షల పదాలకి 150 MB – ఇది ఒక నిజమైన నిర్ణయం

ఒక సాధారణ container lo అది మీ మొత్తం heap కావచ్చు. మరియు ఇది ఒక్క నిమిషం పుట్టింది.

కాబట్టి §2 యొక్క మొదటి ప్రశ్న అలంకారం కాదు: "ఉపసర్గ ప్రశ్నలు నిజంగా ఉన్నాయా?"

లేకపోతే మీరు 145 MB ఎక్కువ ఖర్చు చేసి 19.9× నెమ్మది అయ్యారు.

14. Follow-ups – radix tree, Aho-Corasick, DAWG

"150 MB ని తగ్గించొచ్చా?"

Radix tree (compressed trie): ఒకే పిల్ల ఉన్న nodes ని కలిపేయడం. §9 lo సగటు పిల్లలు **1.00** అని కొలిచాను – అంటే చాలా nodes కి ఒకే పిల్ల ఉంది, మరియు అవన్నీ కలిపేయదగినవే. నేను radix tree ని కొలవలేదు, కానీ ఆ 1.00 అనే సంఖ్య ఎందుకు లాభం ఉంటుందో చెబుతోంది.

"పదాలన్నీ స్థిరమైతే?"

DAWG (directed acyclic word graph) – ఉపసర్గలే కాదు, ప్రత్యయాలని కూడా (suffixes) పంచుకోవడం. – ing, -tion, -ness ఒక్కసారే నిల్వ. చేర్చడం/తీసేయడం కుదరదు. నేను దీన్ని కొలవలేదు.

"ఒక పెద్ద text lo వెయ్యి పదాలని ఒకేసారి వెతకాలంటే?"

Aho-Corasick – trie కి "failure links" కలిపి, text ని ఒక్కసారే చదివి అన్ని matches. నేను దీన్ని కొలవలేదు.

"Autocomplete lo ర్యాంకింగ్?"

ప్రతి node lo ఆ ఉపసర్గ కింద ఉన్న అత్యుత్తమ k పదాలని ముందే నిల్వ చేయడం. అప్పుడు traversal అక్కర్లేదు. ఖరీదు: ఇంకా ఎక్కువ memory, మరియు ప్రతి update కి పైకి propagate. **LLD Deep 17** lo autocomplete ని పూర్తిగా కొలిచాను.

"ఈ doc lo నేను ఏమి కొలవలేదు"

నాలుగు: **radix tree**, **DAWG**, **Aho-Corasick**, మరియు **Unicode పదాలు** – నా నిఘంటువు a-z మాత్రమే.

15. ఏమి నేర్చుకున్నాం

1. **Trie lo విలువలు nodes lo ఉండవు – దారిలో ఉంటాయి.** అందుకే "ఇక్కడ ఒక పదం ముగిసింది" అనేది ప్రత్యేకంగా గుర్తుపెట్టాలి.

2. ఆ ఒక్క **boolean** మర్చిపోతే – యాదృచ్ఛిక **strings** మీద **3.78%**, నిజమైన ఉపసర్గల మీద **100.0%**.

3. మీ నిజమైన **inputs** యాదృచ్ఛికం కావు. **Autocomplete** lo ప్రతి **keystroke** ఒక ఉపసర్గ – అంటే మీరు సరిగ్గా **100%** వరుసలో ఉన్నారు.

4. **has** మరియు **startsWith** రెండు వేర్వేరు ప్రశ్నలు, మరియు తేడా ఒక **&& n.end**.

5. **Trie** ఒక మెరుగైన **hash map** కాదు. ఉపసర్గకి **208x** వేగం, కచ్చితమైన వెతుకులాటకి **19.9x** నెమ్మది.

6. ఆ వేగం **30x memory** తో కొన్నది – 2.34 లక్షల పదాలకి **Set** 5 MB, **trie** 150 MB.

7. **Trie** ఒక **tree**, కాబట్టి **అంచులు = nodes – 1**, కాబట్టి సగటు పిల్లలు ఎప్పుడూ **1.00**. అందుకే 26-గళ్ళ **array** ఎప్పుడూ **~96%** ఖాళీ – ఏ పద జాబితా అయినా సరే.

8. 26-గళ్ళ **array** "వేగం కోసం" **1.33x** ఎక్కువ **memory** తీసుకుంటుంది, మరియు a-z బయట ఏ అక్షరానికీ పనిచేయదు.

9. **remove** పదిహేను పంక్తులు, అందులో నాలుగు **load-bearing** – మరియు దారిని కింది నుంచి తుడవాలి.

10. నా **fuzz** రెండు **mutations** ని 0/120 గా వదిలేసింది – ఎందుకంటే నేను **remove** ని ఎప్పుడూ నిజమైన పదాల మీదే పిలిచాను, మరియు **node** లెక్కని తప్పు క్షణంలో చూశాను.

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Trie ఒక ప్రశ్నకి **208x** వేగం మరియు మిగతా అన్నిటికీ **30x** ఖరీదు. కాబట్టి "trie వాడాలా?" అనే ప్రశ్నకి జవాబు **code** lo లేదు – అది మీ **queries** lo ఉంది.

ఉపసర్గ ప్రశ్నలు ఉన్నాయా? అయితే **trie**, మరియు **end** గుర్తు మర్చిపోకండి.

లేవా? అయితే **Set** . **145 MB** అదా, మరియు **19.9x** వేగం.